

Reinforcement learning to play tic-tac-toe

Q-learning is a brilliant and fundamental method of reinforcement learning that has shown a lot of success recently thanks to the deep learning revolution. Although this tutorial will not explain what is known as deep Q-learning, we will go through the original Q-learning algorithm to teach an agent how to play a game of tic-tac-toe. Despite its simplicity, we will see that it can give excellent results.

First, we will go through some of the necessary background knowledge to get a quick overview of reinforcement learning, then we will present the Q-learning algorithm and lastly how to implement it to teach an agent to play tic-tac-toe.

Overview of reinforcement learning

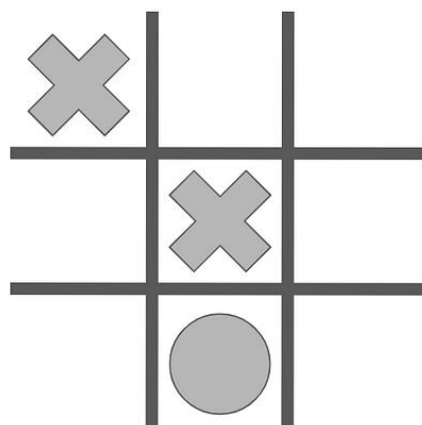
The goal of reinforcement learning is to optimize an agent's behaviour according to some reward function when it is operating in an environment of different states. For this tutorial, the environment is the game of tic-tac-toe which has clearly defined actions and the agent has to decide what actions to pick to win the game. Moreover, the agent will get rewards for winning the game which encourages it to learn good strategies in the game.

A common framework for reinforcement learning is the (finite) Markov Decision Process (MDP). It helps us define a set of actions and states on which the agent's decision-making is based.

An MDP consists of

- A finite set of actions A (position to place a mark on the game board)
- A finite set of states S (a given configuration of the game board)
- A reward function $R(s,a)$ that returns a value based on performing action a in state s .
- A transition function $T(s,a,s')$

The transition function gives the probability of moving from a state s to s' when performing action a . This is necessary when we are in an environment where there is uncertainty about whether an action always produces the desired outcome. However, in the case of tic-tac-toe, we know exactly what each action will do so we will not be using this.



The state-space consists of all possible game configurations. At each state, we can perform a set of available actions. In this example, there are six possible actions for the current player to perform.

The MDP framework helps us formalize the problem to identify which actions, depending on the current state, will maximize the agent's total reward during the game. The reward function $R(s,a)$ will be very simple:

- If the agent performs an action $\hat{a}$ that wins the game from s , then $R(s,\hat{a}) = 1$.
- Else if the agent makes a mistake and picks the wrong action $\tilde{a}$ such that it loses the game then $R(s,\tilde{a}) = -1$.
- Otherwise, when none of the above happens, the reward is simply $R(s,a) = 0$.

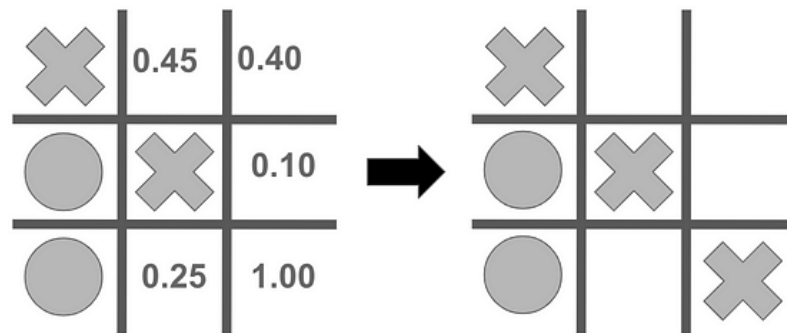
In reinforcement learning, we find to find an optimal policy that the agent uses to decide which actions to pick. Since we are using Q-learning, we will simply denote our policy as the action a that maximizes a function $Q(s,a)$ when the agent is in state s . This is the core of Q-learning, so let's how we would compute this function.

$$a^{best} = \arg \max_{a \in A} Q(s, a)$$

For every state in the game, each action is associated with a Q value. The best strategy is to pick the action that has the highest value. In this example with a state s , the next turn belongs to player X and it is obvious which action has the highest expected reward since X is about to win.

Updating with the Bellman equation

The interpretation of $Q(s,a)$ is that Q is the expected reward given at the end of the game if the agent picks action a in state s . Since the agent wants to maximize its reward, it wants to choose the action that maximizes Q .



Example of a scenario where we display the Q values for the current player X in this state of the game. By placing the next marker where the maximum value lies, the player will win the game and get rewarded.

To compute $Q(s,a)$, the agent has to explore all possible pairs of states and actions while getting feedback from the reward function $R(s,a)$. In the case of tic-tac-toe, we update $Q(s,a)$ iteratively by letting the agent play many games against an opponent. With the Q-learning algorithm, the equation that is used to update Q is the following:

$$Q(s, a)^{new} \leftarrow (1 - \alpha) \cdot Q(s, a) + \alpha \cdot (R(s, a) + \gamma \cdot \max_{\hat{a} \in A} Q(\hat{s}, \hat{a}))$$

- We perform an action a in the current state s .
- The last term looks into the future and returns the largest Q value that is available, $\hat{s}$ is the state that is the new state after performing action a . Then, action $\hat{a}$ is the best action in the next state $\hat{s}$.
- The learning rate α decides to what extent we overwrite the old value, we will use $\alpha = 0.1$.

- The discount factor γ decides how much future rewards should be weighted compared to rewards that are present at the current time step t . We will be using $\gamma = 0.9$.

This equation is based on the well-known equation within reinforcement learning called the Bellman equation. Let's see how to use this equation to teach our agent.

Implementation of Q-learning algorithm

To get a trained agent, we need to learn the values of **$Q(s,a)$** . This will be done by letting two agents play against each other. We will introduce a probability ϵ that each agent picks a random action, otherwise, it will pick the best action according to **$Q(s,a)$** . In this way, we ensure to balance the learning such that the agents sometimes explore new actions and other times exploit the information that the agent already has learnt.

This training phase can be described by the following pseudocode of the Q-learning algorithm:

```
Initialise:  $Q(s,a) = 0$ , starting state  $s$ ,
           starting player  $P$ , iterations  $N$ 
for  $t = 0 : N$ 
    With probability  $\epsilon$  :  $P$  picks random
    action  $a$ 
    Else, pick action  $a$  that maximize  $Q(s,a)$ 
    Observe new state  $\hat{s}$  and reward  $R(s,a)$ 
    If current player is our agent,
        update  $Q(s,a) = (1-\alpha)Q(s,a) + \alpha[R(s,a) + \gamma \max_a Q(\hat{s},a)]$ 
         $s = \hat{s}$ 
    Switch turn,  $P =$  the other player
```

Notice that the number of iterations N must be relatively large, I have been using around 500,000. Also, $Q(s,a)$ can be implemented as a Python dict, or as a two-dimensional array if we represent (s,a) as integers. Lastly, it is possible to vary the probability ϵ over time to emphasize more random exploration in earlier iterations which might speed up the learning.

After you have trained your agent with the algorithm, you may save the values of $Q(s,a)$ and load it when you want to play against the agent. Then, the agent just has to follow the optimal policy by picking actions that maximize **$Q(s,a)$** . Although the agent does not become very intelligent because of how simple the game of tic-tac-toe is, it is still fun to try this out if you want to learn how to implement Q-learning and see that it works.

In summary, first explained the Markov Decision Process (MDP) framework and how it is used in reinforcement learning. We have modelled the game of tic-tac-toe using states, actions and a reward function. On top of that, the function **$Q(s,a)$** is defined that quantifies the expected reward by picking action a in state s , and showed the formula to compute **$Q(s,a)$** by repeatedly playing the game.

Source: <https://towardsdatascience.com/how-to-play-tic-tac-toe-using-reinforcement-learning-9604130e56f6>